PRACTICAL 5

Aim :

Connect the Task Management Express backend to MongoDB using Mongoose. Define a validated Task schema and implement persistent CRUD operations using MongoDB instead of an in-memory array. The application must validate task data, return structured JSON errors, and preserve task records after restarting the server.

Code :

backend/.env :-

```
MONGO_URI=mongodb://127.0.0.1:27017/awd_task_manager

PORT=5500
```

backend/config/db.js :-

```
const mongoose = require('mongoose');


async function connectDB() {
  if (!process.env.MONGO_URI) {
    throw new Error('MONGO_URI is not configured. Add it to backend/.env.');
  }

  await mongoose.connect(process.env.MONGO_URI);
  console.log('MongoDB connected');
}


module.exports = connectDB;
```

backend/models/Task.js :-

```
const mongoose = require('mongoose');
```

```
const taskSchema = new mongoose.Schema({
 title: {
  type: String,
  required: [true, 'Task title is required'],
  trim: true,
  minlength: [2, 'Task title must be at least 2 characters']
 },
 description: {
  type: String,
  trim: true,
  default: ''
 },
 completed: {
  type: Boolean,
  default: false
 },
 priority: {
  type: String,
  enum: ['low', 'medium', 'high'],
  default: 'medium'
 },
 createdAt: {
  type: Date,
  default: Date.now
 }
```

```
}, { versionKey: false });


module.exports = mongoose.model('Task', taskSchema);
```

backend/routes/tasks.js :-

```
const express = require('express');
const mongoose = require('mongoose');
const Task = require('../models/Task');

const router = express.Router();

function asyncHandler(handler) {
  return (req, res, next) =>
    Promise.resolve(handler(req, res, next)).catch(next);
}

function ensureValidId(req, res, next) {
  if (!mongoose.isValidObjectId(req.params.id)) {
    return res.status(400).json({ error: 'Invalid task id' });
  }
  next();
}

router.get('/', asyncHandler(async (req, res) => {
  const tasks = await Task.find().sort({ createdAt: -1 });
  res.status(200).json(tasks);
}));

router.get('/:id', ensureValidId, asyncHandler(async (req, res) => {
  const task = await Task.findById(req.params.id);
  if (!task) return res.status(404).json({ error: 'Task not found' });
  res.status(200).json(task);
}));

router.post('/', asyncHandler(async (req, res) => {
```

```
  const task = await Task.create(req.body);
  res.status(201).json(task);
}));

router.put('/:id', ensureValidId, asyncHandler(async (req, res) => {
  const task = await Task.findByIdAndUpdate(req.params.id, req.body, {
    new: true,
    runValidators: true
  });
  if (!task) return res.status(404).json({ error: 'Task not found' });
  res.status(200).json(task);
}));

router.delete('/:id', ensureValidId, asyncHandler(async (req, res) => {
  const task = await Task.findByIdAndDelete(req.params.id);
  if (!task) return res.status(404).json({ error: 'Task not found' });
  res.status(200).json(task);
}));

module.exports = router;
```

backend/server.js :-

```
require('dotenv').config();

const express = require('express');
const cors = require('cors');
const path = require('path');
const connectDB = require('./config/db');
const taskRoutes = require('./routes/tasks');

const app = express();
const PORT = process.env.PORT || 5500;

app.use(cors());
app.use(express.json());

function requireJsonContent(req, res, next) {
```

```
  if (req.method === 'POST' || req.method === 'PUT') {
    const contentType = req.headers['content-type'] || '';
    if (!contentType.includes('application/json')) {
      return res.status(415).json({
        error: 'Content-Type application/json required'
      });
    }
  }
  next();}
app.use('/api/tasks', requireJsonContent, taskRoutes);
connectDB().then(() => {
  app.listen(PORT, () => {
    console.log(`Server running on port ${PORT}`);
  });
});
```

Testing :

Run the backend from the backend folder using npm run dev. Test the following API requests in Postman.

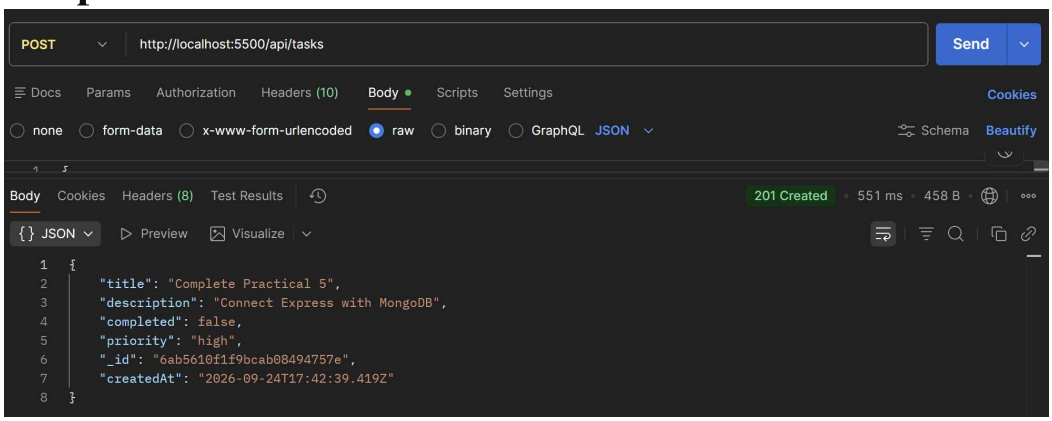
POST /api/tasks body :-

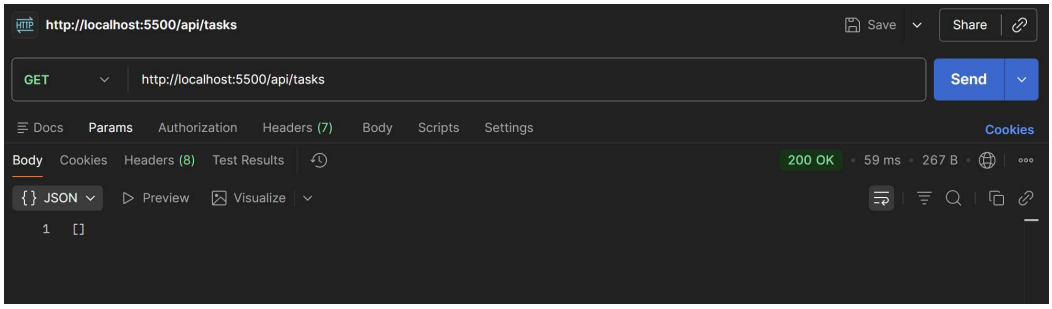
```
{
  "title": "Complete Practical 5",
  "description": "Connect Express with MongoDB",
  "priority": "high"
}
```

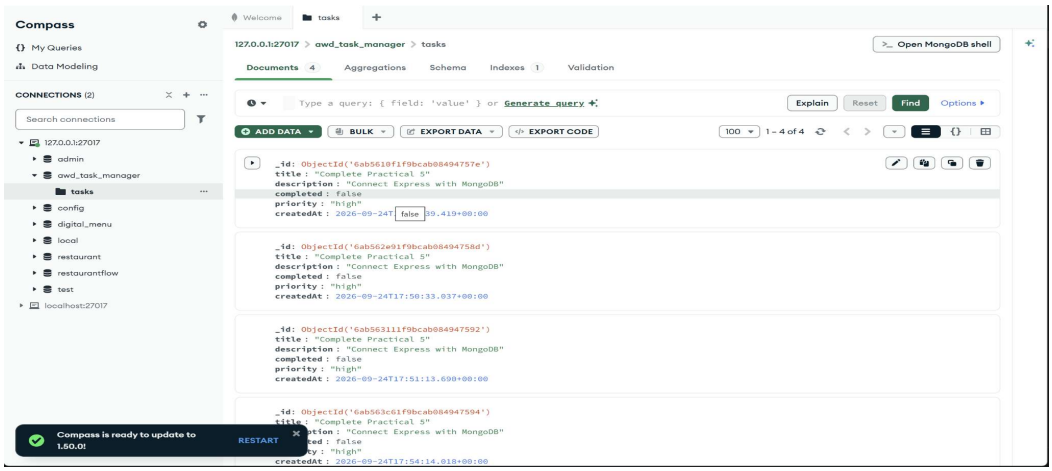
PUT /api/tasks/:id body :-

```
{
  "completed": true
}
```

Output :







Result :

The Task Management backend was successfully connected to MongoDB using Mongoose. A validated Task schema was created with required fields, enum values, and default values. The application successfully performed create, read, update, and delete operations, and task data remained available after restarting the server.